

Foveated rendering

Does it work?
How do we do it?
Why is it hard?
Let's find out!

This slide deck was a presentation I did. It wasn't so much about foveated rendering, it was mostly about aliasing. Read it the same way: have an idea what foveated rendering is, but understand aliasing at the end of it

What is foveated rendering?

Slide 3



We can see quite a large image with our eyes, but only a tiny part of that is in high details

High detail small part : foveal region

Peripheral: vision on the side of your focus, low detail

Foveated rendering renders in high detail in the foveal region, and in less detail in the peripheral region

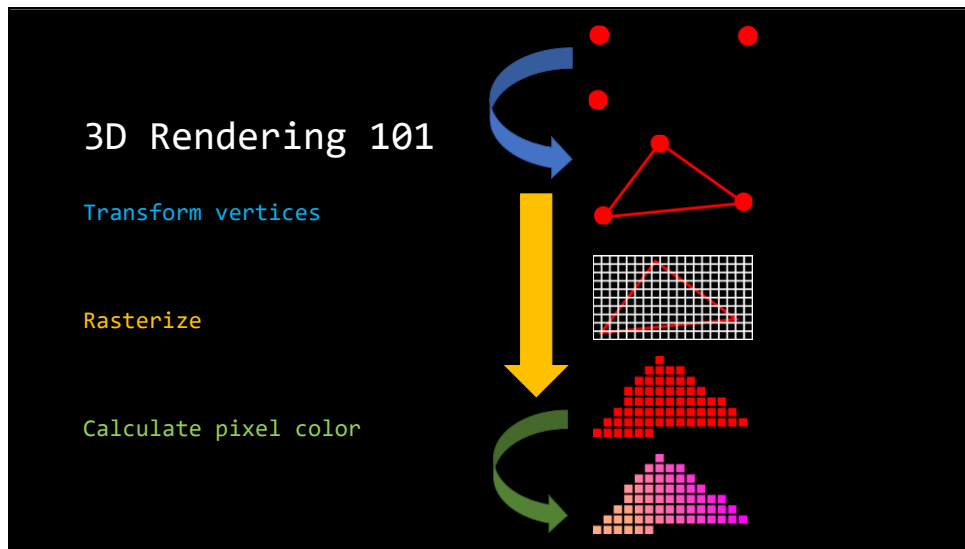
How we know where the user is looking? A device called an eye tracker

Rendering an image
using knowledge of the user's gaze
to use less resources
with no perceived reduction in quality

Gaze is a fancy word for the point on a screen where the user is looking at

Slide 5



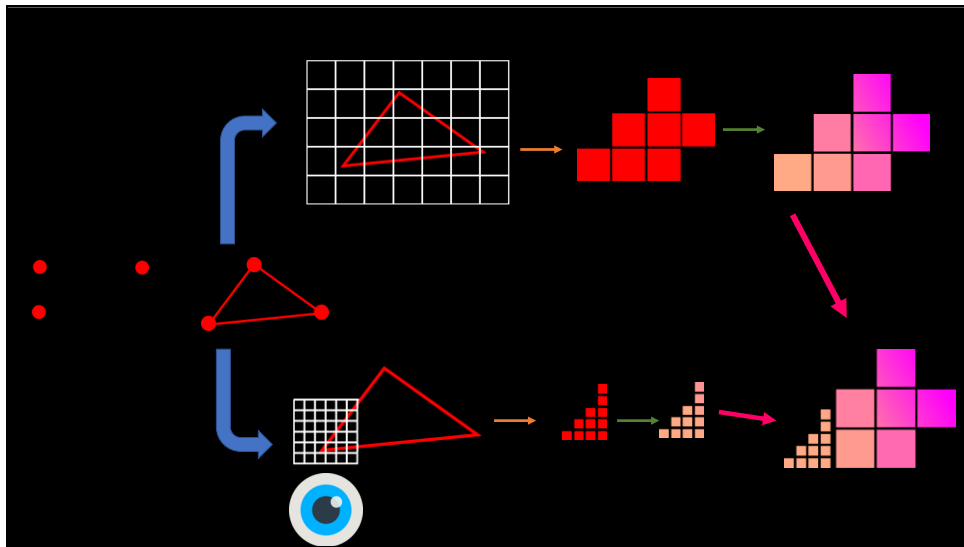


Slide 7

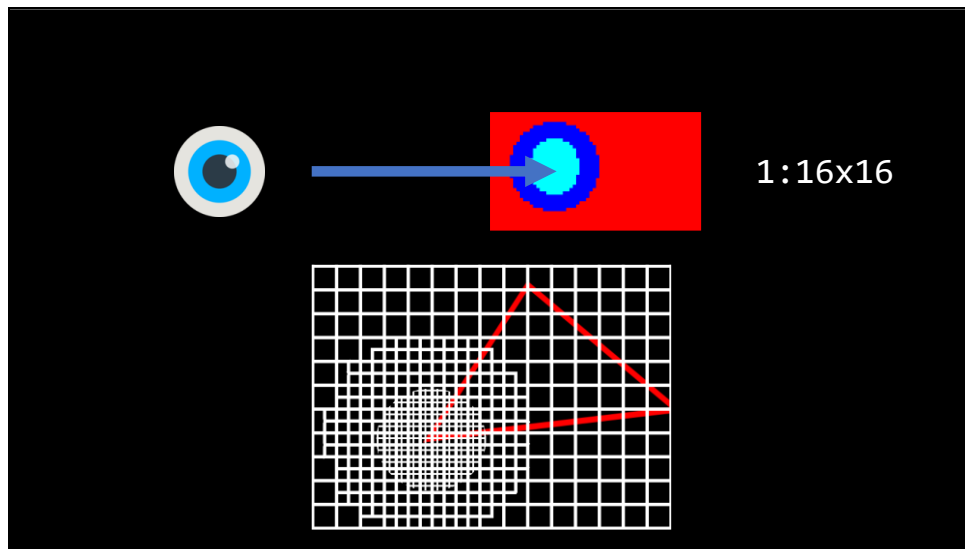


High level algorithm for doing foveated rendering without hardware support

Slide 8



This is the same as the previous slide, showing just one triangle



This is a way simpler way of doing it, but needs hardware support (VRS)

With VRS, we give the GPU a “quality” texture, saying how many samples to take in an area of the screen. We render normally (once) and it just works

VRS

- Set quality levels



- Render normally





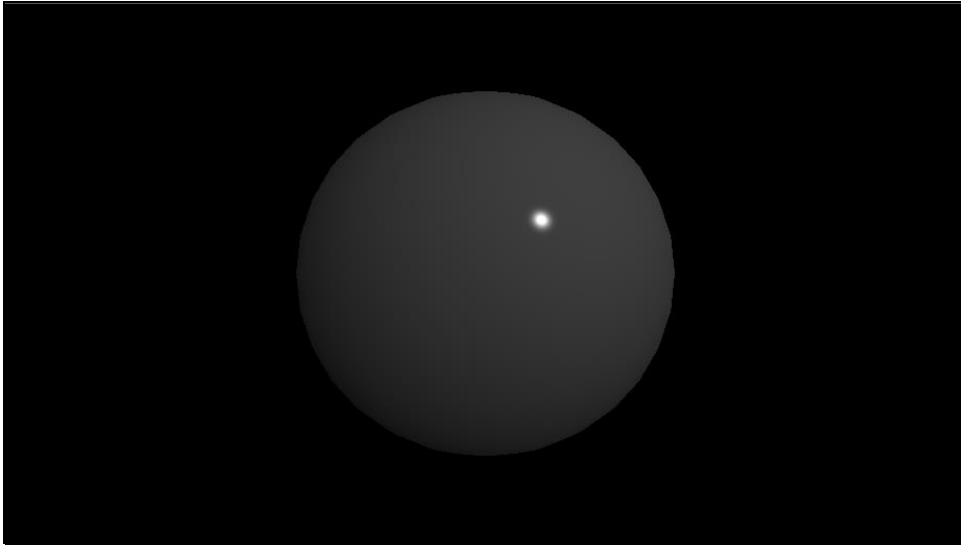
Foveated rendering means less pixels to render, which means it's faster (the whole point of doing foveated rendering)

But it has a major problem that I've struggled with for years: aliasing

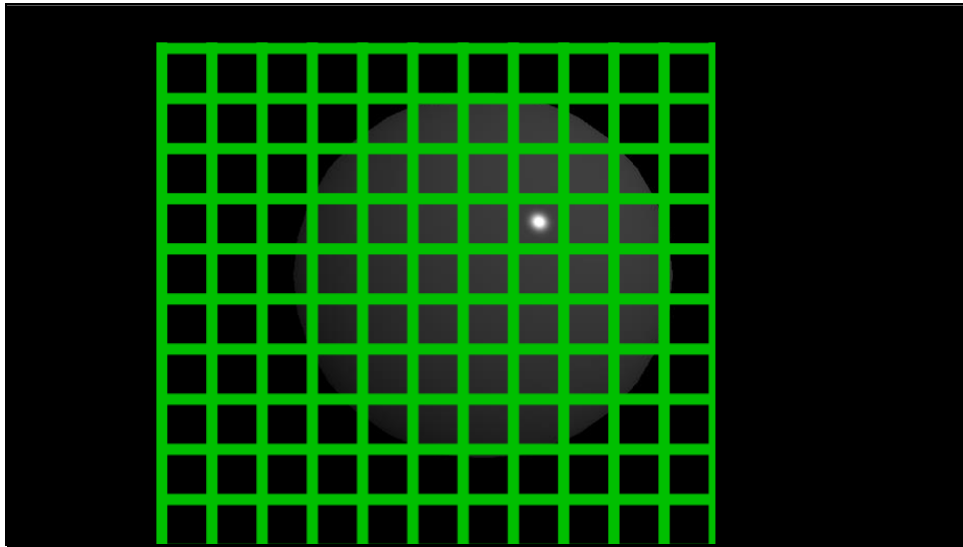
Slide 12



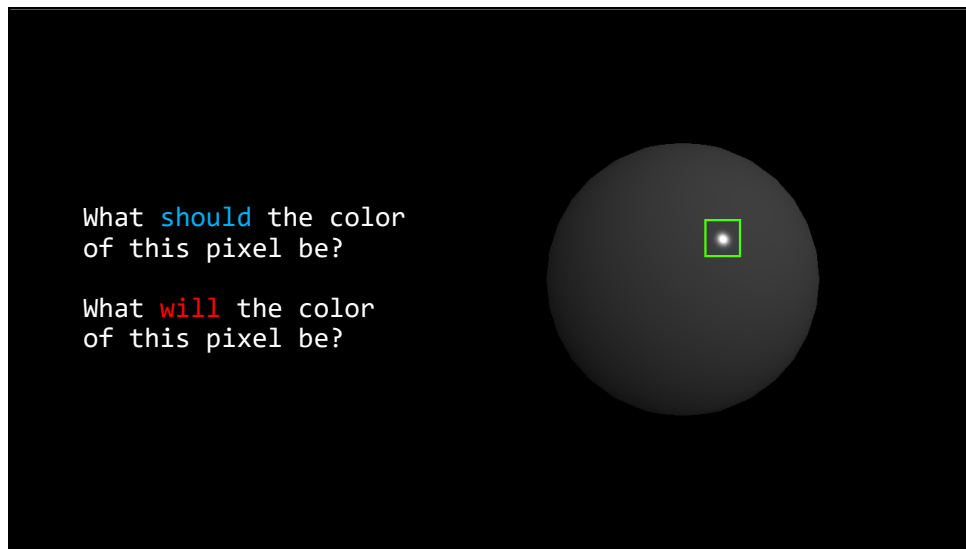




Like the helicopter before, look at this sphere as a sphere in the real world. It's not rendered, it doesn't have a resolution. It's the thing we're going to render



The green grid represents the pixels we're rendering. They're really big (we'll render our sphere to a 10x10 grid)

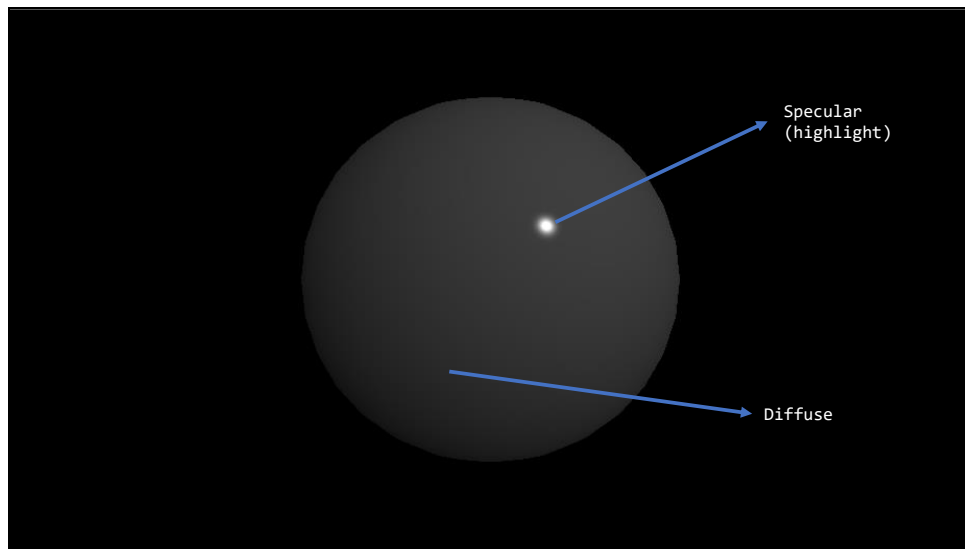


What **should** the color
of this pixel be?

What **will** the color
of this pixel be?

So, see the green square as a single pixel once we're done rendering

Try to answer the questions for yourself, we'll see the answer in the next few slides

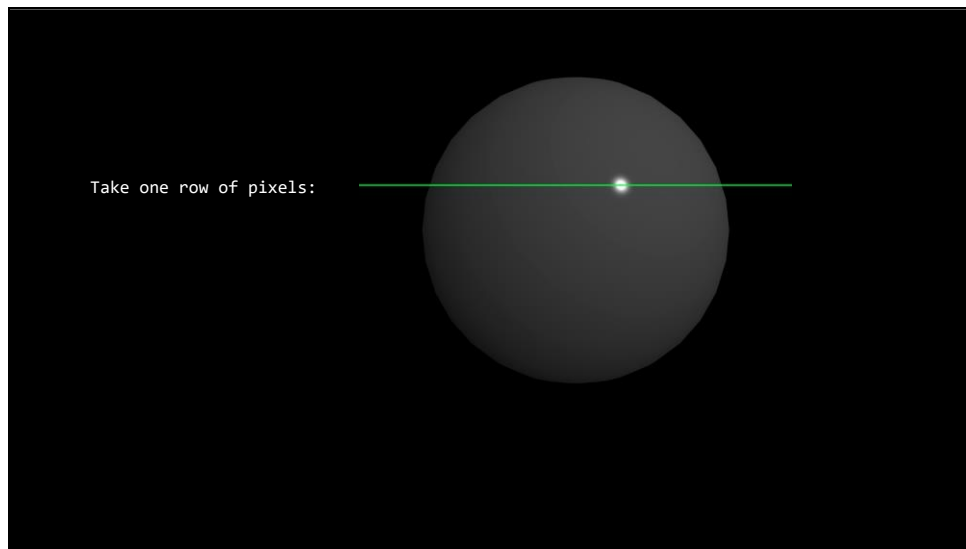


Quick recap:

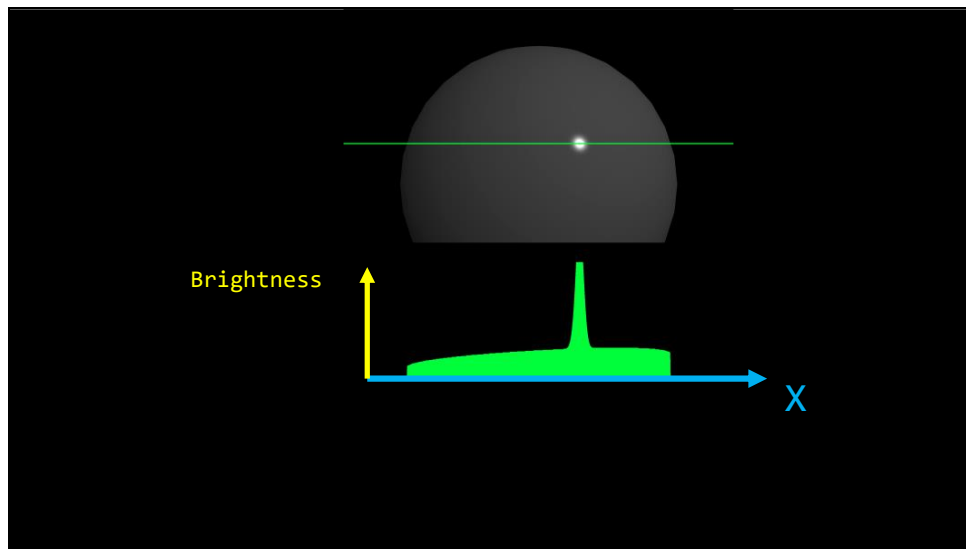
The entire sphere is a dull grey due to diffuse lighting (lighting of unglazed clay)

The small white spot is a specular highlight. Specular is the mirror-like lighting of glazed clay, smooth metal. It moves when you move your head

Slide 18

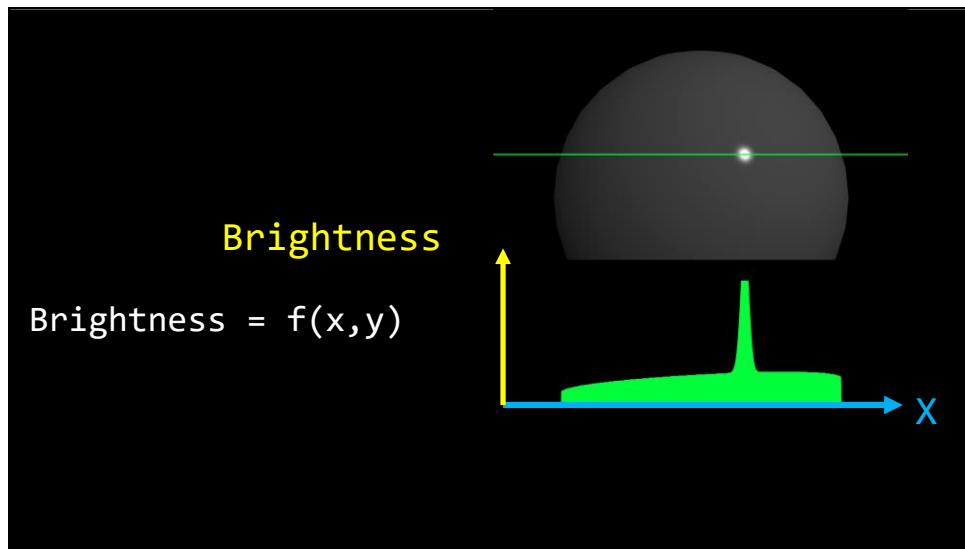


Pixels is a bad word, what I meant is “let’s look as a row of color values along the sphere”



Make sure you understand this

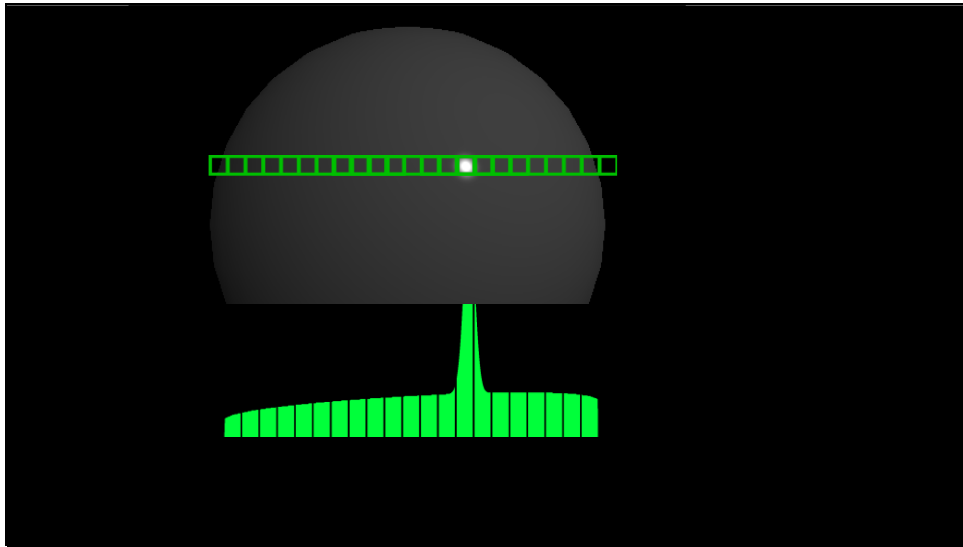
We took a row of color values of the sphere (it's all black/white, so it's only a brightness value) , and we made a graph of this. For each point on the green line above, you'll see it's brightness graphed below



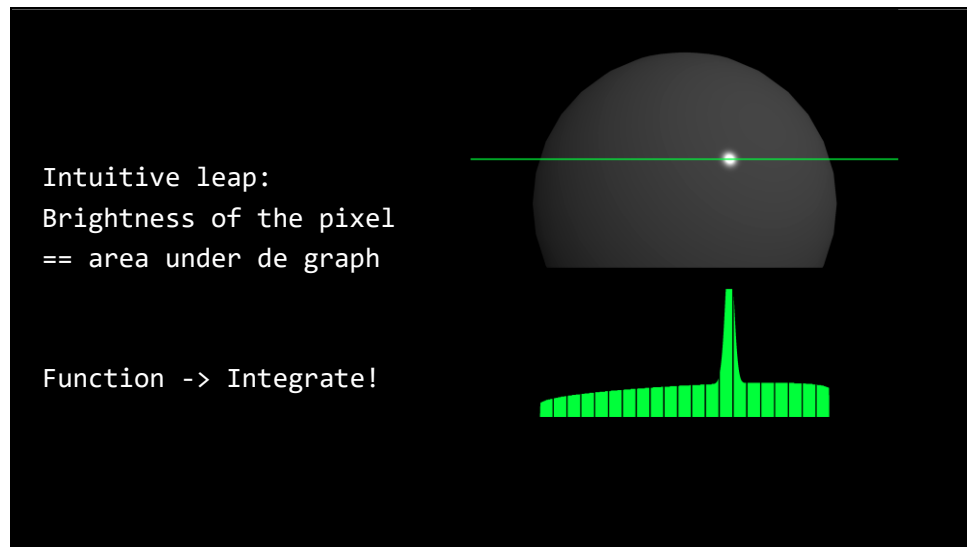
This is just a fancy way of saying the brightness of a pixel can be expressed as a function

You already know this from the first class : brightness = $\text{dot}(N,L)$

Slide 21



All I've done is overlayed the pixel grid over that horizontal line

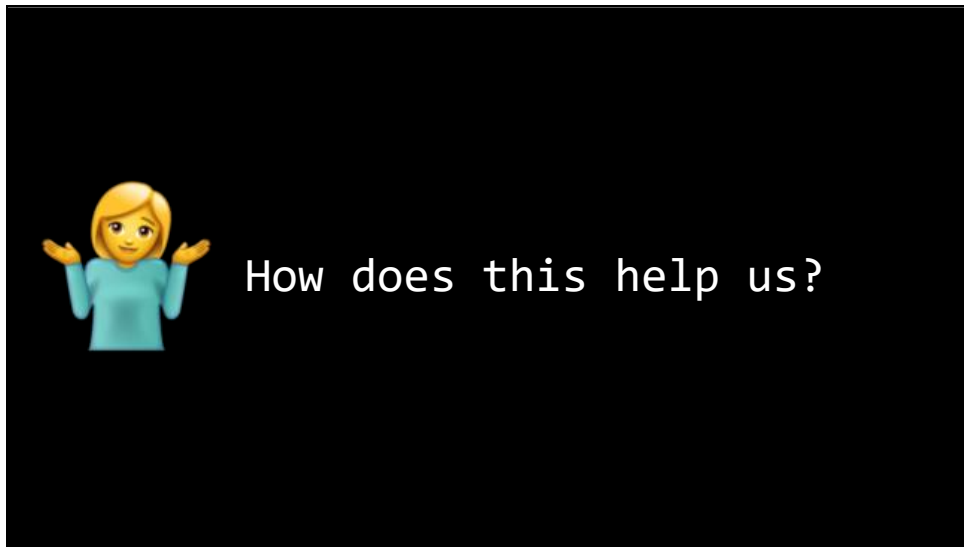


That intuitive leap is quite the leap

Brightness = amount of light leaving the surface and hitting our eye

So, the brightness of a pixel is the amount of brightness “under” the pixels

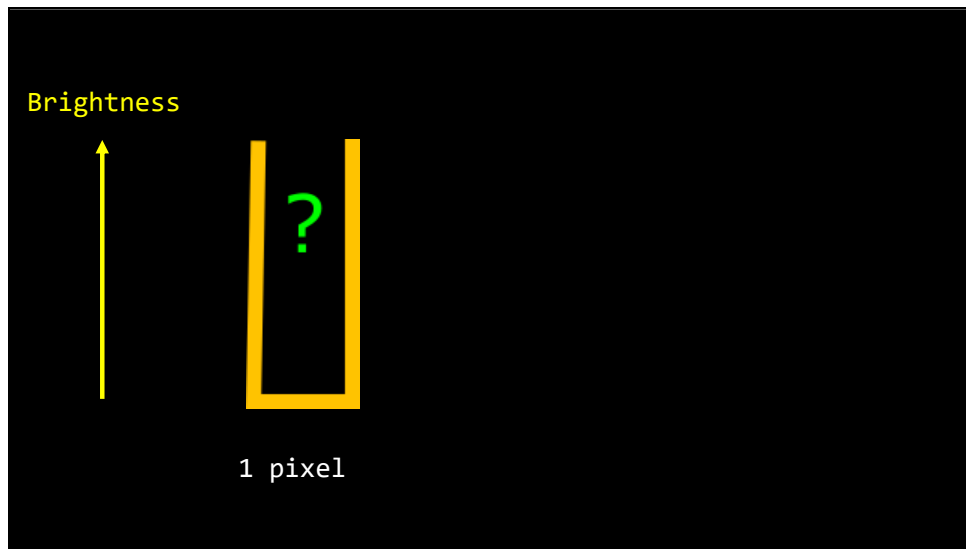
Integrate means calculating the amount of green under a pixel



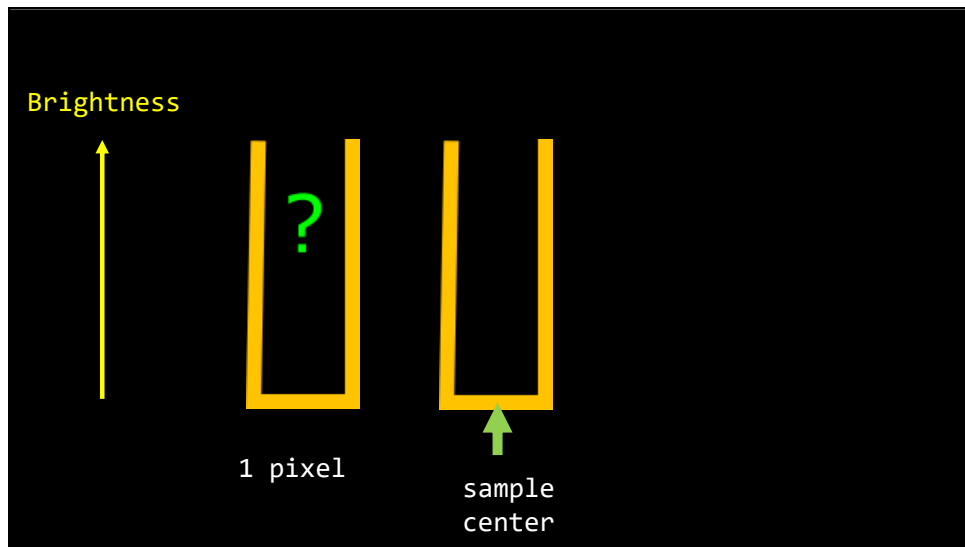
So we need to integrate over a pixel, which means calculating many values of brightness under a pixel

This doesn't happen, it would be too expensive

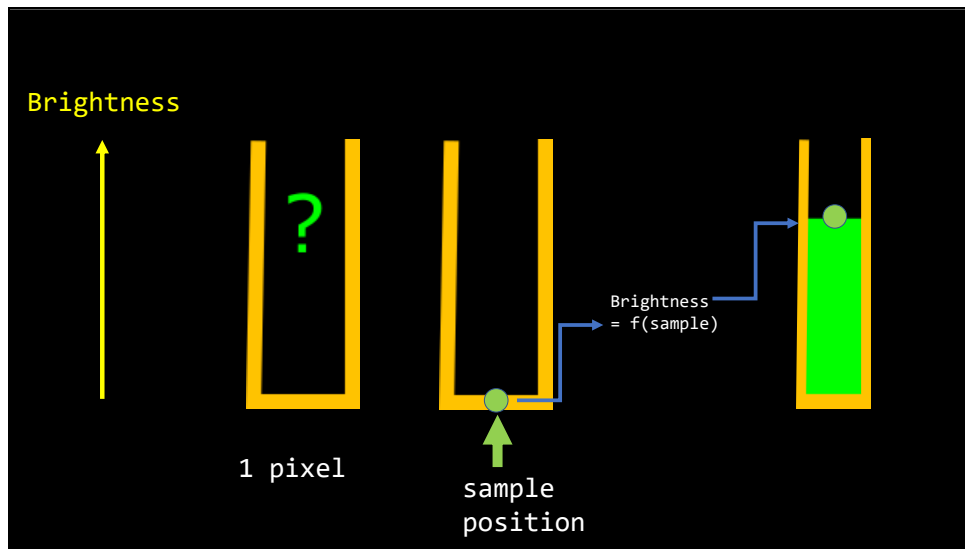
The next slides will explain how rasterization determines what the color will be



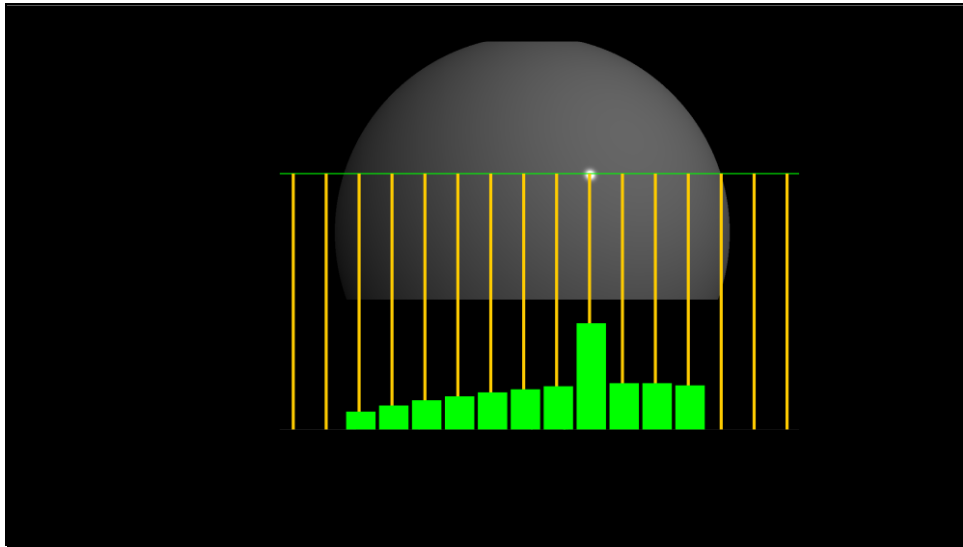
So, what is the brightness of our pixel?



We just check the center of the pixel

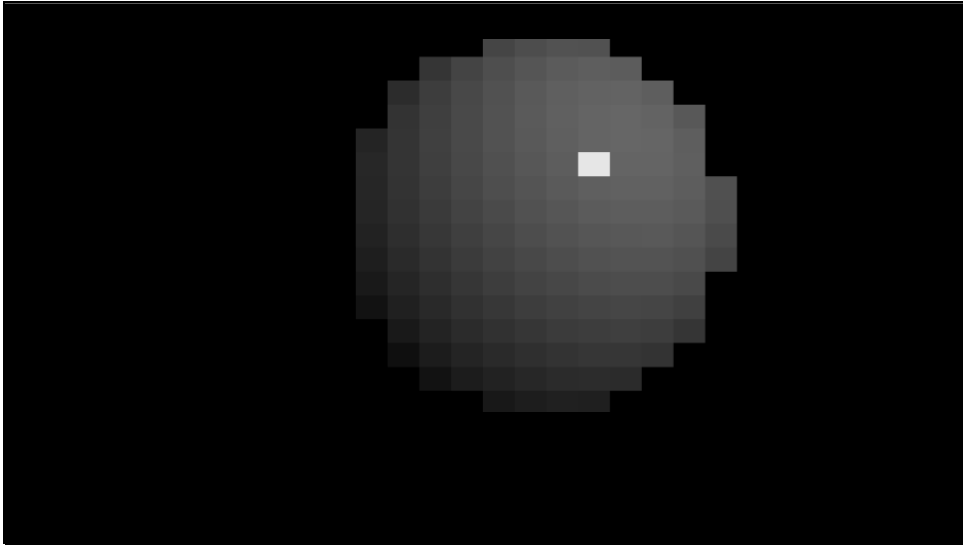


It's not correct, and we'll visually see why next



I've plotted the centers of the pixels here. We see that on the bottom (values of the rendered image), we do show the specular highlight

Slide 28



It's ugly, but it looks like the original

